

Projektarbeit: Computergrafik

Pirmin Pfeifer

1 Einleitung

In diesem Semester wird die Klausur im Kurs **Computergrafik** durch ein Projekt ersetzt. Ziel dieses Projekts ist es, die erlernten Konzepte praktisch anzuwenden und ein eigenes grafisches System zu entwickeln.

Das Projekt kann alleine oder in einer Gruppe von bis zu **drei Personen** bearbeitet werden. Jede Gruppe muss dabei eine eigenständige Implementierung vorlegen. Um das Projekt erfolgreich abzuschließen und die Mindestnote (**4.0**) zu erhalten, müssen bestimmte **Mindestanforderungen** erfüllt werden. Diese Anforderungen sind verpflichtend und garantieren das Bestehen des Kurses.

Zusätzlich gibt es **optionale Erweiterungen**, mit denen weitere Punkte zur Verbesserung der Note erworben werden können. Durch die Umsetzung dieser optionalen Aufgaben kann eine bessere Bewertung erzielt werden, da sie weiterrührende Konzepte der Computergrafik abdecken und eine anspruchsvollere Umsetzung erfordern.

Die genaue Beschreibung der Mindestanforderungen sowie der optionalen Erweiterungen folgt in den nächsten Abschnitten.



Beispiel für einen Rendering Prototype.

2 Bewertungsschema

Auf das Projekt können maximal 100 Punkte erzielt werden. Das Erfüllen der Mindestanforderungen gibt hierbei 50 Punkte, unabhängig von der Teamgröße. Für die Implementierung optionaler Aufgaben können bis zu 50 weitere Punkte P_{opt} erreicht werden. Die Punkte der optionalen Aufgaben werden abhängig von der Teamgröße noch mit einem Teamfaktor f_{team} multipliziert.

Teamgröße	Teamfaktor (f_{team})	Teamfaktor (Dezimal)
1	$\frac{50}{20}$	2.5
2	$\frac{50}{35}$	1.42
3	$\frac{50}{50}$	1

Zusätzlich behält sich der Prüfer vor, einen Präsentationsfaktor $f_m \in [0, 1]$ festzulegen, sollte in der mündlichen Prüfung nicht ersichtlich sein, dass die Implementierung selbstständig durchgeführt wurde.

Die Gesamtpunktzahl ergibt sich somit aus folgender Formel:

$$(50 + (P_{opt} \cdot f_{team})) \cdot f_m$$

3 Abgabe

Die Abgabe des Projekts erfolgt durch das Hochladen der finalen Version auf Moodle als zip-Archiv. Diese muss eine Readme, den Source Code und eine Executable des Projekt beinhalten. Zusätzlich können sie einen Link zu einem Git-Repository auf dem sie Arbeiten abgeben. Der späteste Zeitpunkt des Uploads des Projekts ist **23:59 UTC+2** am Tag vor der mündlichen Prüfung.

4 Mündliche Prüfung

Nach der Abgabe des Projekts finden Mündliche Einzelprüfungen von voraussichtlich jeweils 10-15 Minuten oder als Gruppen Presentation mit 5 Minuten Einzelprüfung statt.

5 Versionshinweis

Dieses Dokument ist die Version **1.0.0**. Bitte überprüfen sie in Moodle ob eine aktuellere Version mit höherer Versionsnummer existiert. Änderungen, mit zum Beispiel Klarstellungen, können jederzeit vorgenommen werden.

6 Mindest Anforderungen

6.1 Rasterisation Pipeline

Implementiere eine Rasterisation Rendering Pipeline mit OpenGL um Geometrie darzustellen. Dies umfasst, das erstellen eines Fensters, das initialisieren des Backends, das anlegend der Rasterisation Pipeline und eine Update-Loop. So wie das deinitialisieren des Backends bei beenden der Anwendung. Die Rendering-Pipeline muss zudem einen Depth-Test beinhalten.

Guides und Ressourcen:

- Learn OpenGL - Getting Started - OpenGL, Creating a window, Hello Window, Hello Triangle.
- Learn OpenGL - Depth testing

6.2 Scene Graph

Der Prototype muss einen Scene Graph zum Rendern verwenden. In diesem muss auch die zum rendern verwendet Camera befinden.

6.3 Model Loading

Der Prototype muss 3D-Modelle laden und rendern können. Nutzen sie hierfür eines der gängigen Formate, wie gltf, glb oder obj. Zusätzlich Informationen aus den Model-Dateien, wie Material Informationen oder Skinning Informationen müssen nicht geladen oder verwendet werden. Laden sie mindestens drei verschiedene 3D Modelle an verschiedene Positionen in ihre Scene. Ein Userinterface um beliebige 3D Modelle zur Laufzeit zuladen und zu rendern ist nicht nötig.

Guides und Ressourcen:

- tinygltf loader
- tinyobjloader
- The Stanford 3D Scanning Repository - Offene Model Bibliothek mit Standard Modellen der Computergrafik

6.4 Phong Shading

Der Prototype muss die 3D Modelle mit dem Phong Beleuchtungsmodell anzeigen können. Nutzen sie hierfür eine Globale Lichtrichtung. Sollten sie für eines der optionalen Aufgaben eine komplexeres Lichtmodell implementieren, gilt diese Anforderung als erfüllt.

Guides und Ressourcen:

- Learn OpenGL - Basic Lighting
- Phong Shading (wikipedia)

6.5 Orbital Camera

Der Nutzer muss in der Lage sein, die Camera um die 3D modelle zu rotieren. Auch das Ändern der Distanz der Camera zum mittelpunkt der Scene muss möglich sein. Fügen sie zudem einen Button im Userinterface hinzu, mit dem sich die Camera Position und Rotation zurück auf die ursprünglichen Werte zurück setzten lässt. Die Camera muss eine perspektivische Projektion verwenden.

6.6 User Interface

Der Prototype muss ein Userinterface bereitstellen. In diesem muss die Globale-Lichtrichtung kontrollierbar sein und zwischen normalen Rendering und einem WireFrame-View umgeschaltet werden können.

Guides und Ressourcen:

- Zur Umsetzung des User Interface wird ImGui empfohlen.

6.7 Dokumentation

Das Projekt muss dokumnetiert werden. Sowohl mit Kommentaren in Code, als auch mit einem README Document, dass die erfüllten optionalen Aufgaben beschreibt. Wird das Projekt als Team bearbeitet, muss zudem die Aufgabenverteilung dokumentiert werden. Insbesondere wer and welchen optionalen Aufgaben gearbeitet hat. Einen Überblick über das gesamte Projekt wird von jedem Teammitglied erwartet, unabhängig davon welche Aufgaben primär bearbeitet wurden.

7 Optionale Aufgaben

7.1 Shading (20P)

7.1.1 Normal Map (5P)

Implementieren Sie Normal Mapping, um detaillierte Oberflächenstrukturen ohne zusätzliche Geometrie darzustellen. Nutzen Sie diese Technik, um realistischere Materialeffekte durch verbesserte Beleuchtung zu erzeugen.

Guides und Ressourcen:

- Learn OpenGL

7.1.2 Alpha Mask (5P)

Ermöglichen Sie Alpha Masking, um teile eine Texturen nicht darzustellen. Nutzen Sie diese Technik für Objekte wie Blätter, Büsche oder Bäume. Es ist ausreichend Alpha Clipping einzubauen.



Beispiel für Alpha Clipping/Masking.

7.1.3 Point Lights (5P)

Fügen Sie Punktlichter hinzu, um realistische Beleuchtungseffekte zu erzeugen. Berücksichtigen Sie Lichtausbreitung für eine verbesserte Darstellung. Schatten sind Teil der der Optionalen Aufgabe 8.5 und dürfen für diese Aufgabe vernachlässigt werden.

Guides und Ressourcen:

- Learn OpenGL

7.1.4 Cube Map Skybox (5P)

Erstellen Sie eine Skybox basierend auf einer Cube Map, um eine realistische Umgebungsdarstellung zu ermöglichen. Verwenden Sie hochauflösende Texturen für eine überzeugende Hintergrunddarstellung. Sie können die Skybox auch als Lichtquelle für den Ambienten Anteil ihrer Beleuchtung verwenden.

Guides und Ressourcen:

- Learn OpenGL
- opengameart cloudy skyboxes
- opengameart retro skyboxes
- polyhaven hdris - manuelle konvertierung in eine Cubemap ist nötig. Sie können panorama-to-cubemap verwenden.

7.2 Raytracing (25P)

7.2.1 Raytracing Spheres (5P)

Implementieren Sie einen Raytracer zur Darstellung von Kugeln. Berechnen Sie Schnittpunkte entweder in einem FragmentShader oder ComputeShader. Die Shattierung der Kugeln sollte der Qualität der Beleuchtung der rasterierten Objekte entsprechen.



Beispiel für eine mit Raytracing dargestellte Kugel.

7.2.2 Raytracing Triangle Geometrie (5P)

Erweitern Sie den Raytracer um die Unterstützung für Dreiecksgeometrien. Laden sie die Geometrie auf die Graphikkarte in eine Format das sie verwednen wollen. Für die erfüllung dieser Aufgabe genügt es alle Dreiecke für jeden Ray zu prüfen.

7.2.3 Raytracing Triangle Geometrie - Accellerated (15P)

Advanced: Erweitern Sie den Raytracer für Dreiecksgeometrien um eine Beschleunigung Struktur ihrer Wahl. Bauen sie eine Beschleunigungs Struktur auf der CPU laden sie diese auf die GPU und benutzen sie diese dann für beschleunigtes Raytracing der Dreiecksgeometrie. Sie können OctTrees, K-D-Trees oder Bounding-Volume-Hirachien benutzen. Eine fortgeschrittene optimierung der Datenstrukturen ist nicht notwendig, das verwenden von einer diese Datenstrukturen sollte in jedem Fall für eine signifikate Beschleunigung des Raytracing ausreichen.

7.3 Interaktiver Prototyp (25P)

7.3.1 Camera Modes (5P)

Implementieren Sie verschiedene weitere Kameramodi, wie eine 6Degrees-of-Freedom-Camera (Horizontale, Verticale, Laterale-Bewegung, sowie Roll, Pitch und Yaw), um die Navigation innerhalb der Szene zu verbessern. Fügen sie zudem Optionen hinzu das FieldOfView und die Far- und Near-Clipping plane im GUI einzustellen.

7.3.2 Animation Nodes (5P)

Fügen sie verschiedene Animation Nodes hinzu um die Scene zubewegen:

- Stetige Rotation um eine Axse
- Ozilierende Rotation um eine Axse
- Ozilierende Translation entlang einer Axse

Diese Nodes sollen sich wie andere Translation Nodes verhalten, nur dass diese ihre aktuelle Transformation jeden Frame in einer Update oder Animate Funktion ändern.

7.3.3 Scene Editor (10P)

Erstellen Sie einen Szeneneditor, mit dem Benutzer Objekte platzieren, verschieben und konfigurieren können. Integrieren Sie eine intuitive Steuerung für einfache Handhabung. Für das Interface Design können sie sich an gängigen 3D editoren wie Blender, Unity, Unreal Engine oder ähnlicher Software orientieren.

Guides und Ressourcen:

- ImGuizmo - Kollektion an Guizmos und Widgets die für eine Scene editor nützlich sind.

7.3.4 Scene Saving and Loading (5P)

Ermöglichen Sie das Speichern und Laden von Szenen, sodass Nutzer ihre erstellten Umgebungen speichern und später wiederherstellen können.

7.4 Splines (25P)

7.4.1 Spline Renderer (15P)

Implementieren Sie ein Rendering-System für Splines, um gekrümmte Linien und Pfade präzise darzustellen. Nutzen Sie mathematische Interpolationstechniken für eine exakte Visualisierung.

7.4.2 Spline UI (10P)

Erstellen Sie eine interaktive Benutzeroberfläche zur Steuerung von Splines. Ermöglichen Sie Nutzern das Erstellen, Bearbeiten und Anpassen von Kurven innerhalb der Szene. Die erstellten Splines sollten speicherbar sein und wieder geladen werden können.

7.5 Game 5-35P

Sie können auch ein kleines Spiel in ihren Prototype einbauen. Dies muss vorher mit dem Dozenten abgesprochen werden, damit eine Punktzahl (5-25P), im Verhältnis zum Aufwand des Spiels, festgelegt werden kann. Spielideen:

- TicTacToe
- Minesweeper
- Pong
- Tetris
- Connect Four

Implementieren sie die Spiellogik und Visualisierung in ihren Prototype. Das Spiel muss mit Maus oder Tastatur spielbar sein.

Guides und Ressourcen:

- ItchIo - Kollektion an kostenlosen game assets

7.5.1 Sound 5P

Fügen sie Sounds und Musik zu ihrem Spiel hinzu. Es sollten mindest fünf Soundeffekt und eine Hintergrundmusik verwendet werden.

Guides und Ressourcen:

- soloud - Opensource Audio Library
- itchio - free sound effects

7.5.2 Gamepad Support 5P

Fügen sie die option das Spiel mit einem Gamepad zuspieren hinzu. Das Spiel sollte entweder XBox Controller oder Nintendo Switch Pro Controller unterstützen.

7.6 Antialiasing 5P

Fügen sie Multisample-Kantenklättung zu ihrem Prototypen hinzu.

Guides und Ressourcen:

- Learn OpenGL

8 Advanced: Optional Aufgaben

Die "advanced optionalen Aufgaben" beinhalten fortgeschrittene Themen die über die Inhalte der Vorlesung hinaus gehen. Diese Aufgaben sind teilweise nur grob umrissen, für weitere Informationen wenden sie sich gerne an den Dozenten.

8.1 Frustum Culling 10P

Objekte die außerhalb des View-Frustums und somit außerhalb des zuberechnenden Bild liegen tragen nicht zum Ergebnis bei verbrauchen jedoch Ressourcen. Beim Frustum Culling wird für Objekte vor dem Drawcall überprüft ob dies überhaupt im View-Frustum liegen. Implementieren sie Frustum Culling für ihre Rendering Pipeline. Zum überprüfen des Cullings, reicht es aus die Objekte durch ihre Achsenparallele Bounding Box (AABB) zu approximieren. Bauen sie zudem eine größere Scene und zeigen im UI an wie viele Drawcalls aktuell ausgeführt werden und wie viele durch das Culling verhindert wurden.

Guides und Ressourcen:

- [Learn OpenGL](#)
- [Vorlesung - View Frustum Culling - Universität Bremen](#)
- [View Frustum Culling Tutorial - Universität Bremen](#)

8.2 Instanced Draw Calls 15P

Wird das gleiche Model mit dem gleichen Shader mehrfach nur mit Parameter unterschieden gerendered, können wir diese zu einem Instanced Draw Call zusammen fügen. Zählen sie die Anzahl der gleichen Drawcalls in ihrem Scene Graph und fassen sie diese automatisch zu einem einzelnen Instanced Drawcall zusammen.

Guides und Ressourcen:

- [Learn OpenGL](#)

8.3 Skinning and Animation 20P

Fügen sie Skeletal Meshes und Animationen zu ihrem Prototype hinzu. Der Prototype muss verschiedene Animationen auf einem skeletal Mesh abspielen können.

Guides und Ressourcen:

- [Learn OpenGL](#)
- [Mixamo - Free Animated Meshes](#)

8.4 Volumetric Rendering 20P

Implementieren sie Volumetric Clouds oder Volumetric Fog mit Hilfe von Raymarching. Guides und Ressourcen:

- Basic Volumetric Rendering - A Masterclass from GPC24
- Basic Volumetric Rendering Preview
- Real-time dreamy Cloudscapes with Volumetric Raymarching
- The Real-Time Volumetric Cloudscapes of Horizon Zero Dawn
- Nubis: Realtime Volumetric Cloudscapes In A Nutshell

8.5 Shadows 20P

Implementieren sie ShadowMapping für ihre Globale Lichtquelle und/oder Punktlichter.

Guides und Ressourcen:

- Learn OpenGL
- Shadow mapping - Tutorial

8.6 Deferred Rendering 20P

Deferred Rendering ist eine Optimierung um mehrfache Lichtberechnungen zu vermeiden. Hierbei werden zuerst mehrer Framebuffer gerendert, die die nötigen Informationen für die Lichtberechnungen beinhalten (Albedo, Normalen, Specular und Roughness, ...). Anschließend werden diese Framebuffer zusammen kombiniert um das finale Bild zu berechnen, hierbei werden auch alle Lichtberechnungen durchgeführt. Somit wird das Licht nur einmal pro Pixel berechnet.

Guides und Ressourcen:

- Learn OpenGL

8.7 Eigene Idee 5-25P

Sie haben auch die Option eine eigene Idee für eine Aufgabe einzubringen. Wenden Sie sich hierfür einfach an den Dozenten.

9 Versionshistorie

- **1.0.0** - Initale Version für das Sommersemester 2025